

CSC111 Winter 2026 Project 1

Alsade Brianna Daley, Elif Cakici, Zila Okoroze-Trust

February 13, 2026

1 Running the game

The game can be run by executing `adventure.py` using Python, no additional libraries required.

Game Map

Example game map (replace with actual IDs from `game_data.json`):

```
-1  1  2
-1  3 -1
  5  4  7  9 10
-1  6  8 -1 11
```

Starting location is: 1

Game solution

List of commands to win the game:

1. pick up toonie
2. pick up pencil
3. go east
4. pick up usb drive
5. go west
6. go south
7. pick up jam
8. go south
9. go west
10. pick up key fob : answer the riddle with "robarts"
11. go east
12. go south
13. use jam

14. pick up lucky mug
15. go north
16. go east
17. go east
18. use pencil
19. pick up laptop charger
20. go east
21. go south
22. use key fob

Lose condition(s)

Description: The player loses the game if they exceed the maximum allowed number of moves, which is 35.

List of commands that cause a loss are: (example from `lose_demo`):

```
["go south", "go north"] * 17 + ["go south"]
```

Parts of the code involved:

- `adventure.py`, `AdventureGame` class: the `player.moves` attribute tracks the number of moves.
- The game loop checks if moves exceed `MAX_MOVES` and if so, ends the game.
- Helper functions in `adventure.py` assist with validating moves and commands.

Inventory

1. All location IDs that involve items in the game: 1, 2, 3, 5, 6, 8, 9
2. Item data:
 - (a) Item 1:
 - Item name: "toonie"
 - Item start location ID: 1
 - Item target location ID: 4
 - (b) Item 2:
 - Item name: "pencil"
 - Item start location ID: 1
 - Item target location ID: 9
 - (c) Item 3:
 - Item name: "usb drive"
 - Item start location ID: 2
 - Item target location ID: 11
 - (d) Item 4:

- Item name: "jam"
 - Item start location ID: 3
 - Item target location ID: 6
- (e) Item 5:
- Item name: "key fob"
 - Item start location ID: 5
 - Item target location ID: 11
- (f) Item 6:
- Item name: "lucky mug"
 - Item start location ID: 6
 - Item target location ID: 11
- (g) Item 7:
- Item name: "laptop charger"
 - Item start location ID: 9
 - Item target location ID: 11
3. Example commands to pick up and use items (from `inventory_demo`):

```
"pick up toonie",
"pick up pencil",
"inventory",
"go east",
"pick up usb drive",
"inventory"
"drop pencil"
```

4. Code involved in handling the `inventory` command:
- `adventure.py`, the function `handle_menu_command()`
 - `Player` class in `game_entities.py`, methods `pick_up`, `drop_item`, `has_item`
 - Helper functions in `adventure.py` assist with validating item commands

Score

1. Players earn points by depositing items in their target locations. The first location to earn points is location 4. Commands leading to score increase (from `scores_demo`):

```
"pick up toonie",
"go south",
"score"
```

2. Code involved:
- `adventure.py`, `handle_item_command()`, updates `player.score`
 - `handle_menu_command()` displays the current score
 - `Player` class in `game_entities.py`
 - Helper functions in `adventure.py` are used for validating item locations and scoring

Enhancements

1. Player Class Puzzle Enhancement

- Brief description: Added a system for using items to solve interconnected puzzles in the game. Steps include:
 - (a) Give the beggar a `toonie` to receive a hint
 - (b) Solve a riddle to obtain the `key fob`
 - (c) Exchange `jam` or cookies for a `lucky mug` in the dining hall
 - (d) Be tricked into joining a cookie line, losing extra steps
 - (e) Give a pencil to the receptionist to get a charger in fewer steps
 - (f) Use the `key fob` to unlock the dorm door
- Complexity level: Medium
- Justification: Requires significant additions to the baseline game logic, including:
 - Updating the `Player` class to handle `use item` functionality
 - Adding multiple interconnected puzzle steps
 - Modifying `adventure.py` for item interactions
 - Adding new items
 - Adding code to manage various types of wrong inputs
 - Managing game state, unlocked locations, and move count
 - Challenge: Attempted to move helper functions to a separate file, but circular import errors occurred, so logic remains in `adventure.py`
- Code involved:
 - `adventure.py`, `AdventureGame` class, methods handling item interactions and location puzzles
 - `Player` class in `game_entities.py` for inventory and item usage, including interact modules used to manage puzzle locations
 - `handle_command()` and `handle_item_command()` for executing the `use item` commands
 - Helper functions in `adventure.py` used to validate moves, items, and puzzle conditions
- Commands from `enhancements_demo`:

```
"pick up toonie",  
"go south",  
"go south",  
"go west",  
"pick up key fob"
```